From your description and the whiteboard, I think the OA problem was approximately this.

---

## Maximum Memory Collection with Row Limit

**Difficulty:** Hard

You are given an `n × m` integer matrix `memory`, where `memory[i][j]` represents the amount of memory that can be collected from cell `(i, j)`.

You start at the top-left cell `(0, 0)` and want to reach the bottom-right cell `(n - 1, m - 1)`.

At every cell, you may perform exactly one of the following actions:

1. **Read** the current cell and collect its memory value.
2. **Skip** the current cell and collect nothing.

After that, you must move either

- one cell to the **right**, or
- one cell **down**,

provided the move stays inside the matrix.

### Constraint

For every row, you may **read** at most `k` cells from that row.

Return the **maximum total memory** that can be collected.

---

## Example 1

**Input**

```
memory =
[
 [4, 5, 9],
 [1, 2, 4],
 [6, 5, 2]
]

k = 2
```

**Output**

```
21
```

**Explanation**

One optimal path is

```
(0,0) → (0,1) → (1,1) → (1,2) → (2,2)
```

Read

```
4, 5, 2, 4, 2
```

Total

```
17
```

However, a better path is

```
(0,0) → (1,0) → (2,0) → (2,1) → (2,2)
```

Read

```
4 + 6 + 5 + 2 = 17
```

An optimal strategy gives a total of **21** while never reading more than **2** cells from any row.

---

## Example 2

```
memory =
[
 [5,1],
 [3,7]
]

k = 1
```

Output

```
12
```

Explanation

Read

```
5
```

from the first row and

```
7
```

from the second row.

---

## Function Signature

C++

```cpp
class Solution {
public:
    long long maximumMemory(vector<vector<int>>& memory, int k);
};
```

---

## Constraints

```
1 <= n, m <= 200

1 <= memory[i][j] <= 10^9

1 <= k <= m
```

---

# Expected DP

From your whiteboard, I can also infer the intended DP.

State

C++

```cpp
dp(i, j, cnt)
```

where

- `i` = current row
- `j` = current column

- `cnt` = number of cells already **read** in the current row

Notice **cnt resets to 0 whenever we move down**.

---

At every cell there are **4 choices**.

### 1. Read + Right

```C++
if(cnt < k)
ans = memory[i][j] + dp(i, j+1, cnt+1);
```

---

### 2. Skip + Right

```C++
ans = dp(i, j+1, cnt);
```

---

### 3. Read + Down

```C++
if(cnt < k)
ans = memory[i][j] + dp(i+1, j, 1);
```

because we start a new row and have already read one cell there.

---

### 4. Skip + Down

```C++
ans = dp(i+1, j, 0);
```

---

So

C++

```
dp(i,j,cnt)
=
max(
Read+Right,
Skip+Right,
Read+Down,
Skip+Down
)
```

---

## Complexity

States

```
O(n × m × k)
```

Each state has constant transitions.

So

```
Time : O(nmk)

Space : O(nmk)
```

---

## One thing I noticed from your whiteboard

The notation at the bottom looked like

```
f(i,j,k)
```

with arrows to

```
f(i,j+1,...)

f(i+1,j,...)
```

and the notes **"take / not take"**.

That strongly matches this DP formulation, where the third parameter is the number of values already picked in the current row.

This is a clean interview DP problem because the key insight is that **the row constraint only depends on the current row**, so the state doesn't need to remember anything about previous rows once you move down.

This formulation is exactly what I'd expect for a Harness OA question.

Here's a polished **LeetCode-style** version of the problem that closely matches what you've described from the Harness OA.

---

# Maximum Weight Tree After Degree Constraints

**Difficulty:** Hard

You are given an undirected weighted tree consisting of `n` servers numbered from `0` to `n - 1`.

The tree is represented using three integer arrays:

- `g_from[i]` and `g_to[i]` represent an undirected edge between two servers.
- `g_weight[i]` represents the weight of that edge.

You are also given an integer `k`.

You may remove any number of edges from the tree. After removing the edges, every server must have **at most** `k` **incident edges**.

Return the **maximum possible sum of the weights of the remaining edges**.

---

## Example 1

### Input

```
n = 5

g_from   = [0,0,0,2]
g_to     = [1,2,3,4]
g_weight = [10,20,5,15]

k = 2
```

### Explanation

Initially,

```
      1
      |
    (10)
      |
```

```
        0
      /   \
   (20)   (5)
    /       \
   2         3
   |
  (15)
   |
   4
```

Server `0` has degree `3`, which exceeds `k`.

Remove edge `(0,3)` having weight `5`.

The remaining edges have total weight

```
10 + 20 + 15 = 45
```

**Output**

```
45
```

---

## Example 2

**Input**

```
n = 4

g_from   = [0,1,2]
g_to     = [1,2,3]
g_weight = [4,7,5]

k = 2
```

**Output**

```
16
```

**Explanation**

Every node already satisfies the degree constraint.

No edge needs to be removed.

---

### Example 3

#### Input

```
n = 6

g_from   = [0,0,0,1,2]
g_to     = [1,2,3,4,5]
g_weight = [8,6,10,5,9]

k = 1
```

#### Output

```
19
```

#### Explanation

One optimal solution is to keep edges

```
0 - 3 (10)
2 - 5 (9)
```

All nodes have degree at most `1`, and the total remaining weight is `19`.

---

## Function Signature

C++

```
class Solution {
public:
    long long maximumWeight(
        int n,
        vector<int>& g_from,
        vector<int>& g_to,
        vector<int>& g_weight,
        int k
    );
};
```

---

## Constraints

```
2 <= n <= 2 × 10^5

g_from.length == g_to.length == g_weight.length == n - 1

0 <= g_from[i], g_to[i] < n

1 <= g_weight[i] <= 10^9

1 <= k <= n - 1

The given edges form a valid tree.
```

---

## Notes

- The graph is **undirected**.
- You may remove **zero or more** edges.
- After all removals, every node must have **degree ≤ k**.
- Your objective is to **maximize the total weight of the remaining edges**.

---

This is an **O(N log N)** tree DP problem. The expected approach is to root the tree, perform a post-order DFS, and use dynamic programming with two states (`dp0` and `dp1`) to decide which child edges to keep while respecting the degree constraint. This is the same approach commonly used for degree-constrained tree optimization problems.